MongoDB – Types de données, Collections
CRUD MongoDB partie 1

1. Types de données MongoDB

MongoDB stocke les données dans un format **BSON** (Binary JSON), similaire à JSON mais optimisé pour la base de données. Chaque document est un ensemble de **paires clé-valeur**.

Voici les principaux types de données :

1. **String** – Texte ou chaîne de caractères

{ "prenom": "Amina" } // le champ "prenom" contient du texte

2. **Integer** – Nombre entier

{ "age": 19 } // nombre entier, 32 bits ou 64 bits

3. **Double** – Nombre à virgule flottante

{ "moyBaccalaureat": 14.25 } // nombre décimal

4. **Boolean** – Vrai ou faux

{ "admis": true } // champ booléen

5. **Date** – Date et heure

{ "dateInscription": new Date() } // objet Date en JavaScript

6. **Null** – Valeur vide

{ "telephone": null } // champ sans valeur

7. **Binary** – Données binaires (images, fichiers…)

{ "binaryData": "01010101" } // exemple binaire

8. **Array** – Tableau de valeurs (même type ou mixte)

{ "skills": ["Python", "JavaScript", "PHP"] } // liste de compétences

9. **ObjectId** – Identifiant unique généré automatiquement

{ "_id": ObjectId("62dd128a1a19f3d7ecc6624f") } // ID unique MongoDB

2. Collections

- Une **collection** est un **conteneur de documents**, comme une table dans une base relationnelle.
- Elle permet d'organiser les documents par thème.

  Exemple :

db.Stagiaires *// collection "Stagiaires" dans la base de données*

- Les collections sont créées automatiquement lors de l'insertion d'un document si elles n'existent pas encore.

3. Espaces de noms (Namespace)

- Un **namespace** combine le nom de la base et de la collection.
- Forme : nomBase.nomCollection

  Exemple :

myDB.Stagiaires *// base "myDB", collection "Stagiaires"*

CRUD MongoDB – Partie 1

Insertion de documents

1. **insertOne** – Ajouter un document unique

```
db.Stagiaires.insertOne({
  "prenom": "Amina",
  "filiere": "Dev Digital",
  "niveau": "1A"
}) // insère un stagiaire dans la collection
```

insertOne() permet d'ajouter un seul document dans la collection.

Ici, un stagiaire nommé Amina est inséré avec ses informations : prenom, filiere, niveau.

MongoDB génère automatiquement un _id unique pour chaque document.

✔ Insertion avec _id personnalisé

```
db.Stagiaires.insertOne({
  _id: 1,
  prenom: "Amina",
  filiere: "Dev Digital"
})
```

- Par défaut, MongoDB **génère automatiquement un _id unique** pour chaque document.
- Ici, nous **fournissons un _id personnalisé** (_id: 1).
- Cela permet de **contrôler l'identifiant** du document pour des besoins spécifiques (références, exercices, tests…).

2. **insertMany** – Ajouter plusieurs documents

```
db.Stagiaires.insertMany([
  { "prenom": "Nidal", "filiere": "Cyber Security", "niveau": "2A" },
  { "prenom": "Salim", "filiere": "Full Stack", "niveau": "2A" }
]) // insère plusieurs stagiaires en une seule commande
```

insertMany() permet d'ajouter **plusieurs documents à la fois** dans une collection.

Très pratique pour **préparer rapidement des données de test ou un jeu de données complet**.

MongoDB génère automatiquement un _id unique pour chaque document si aucun _id n'est fourni

NB : Toujours vérifier les documents insérés avec **find**().

✔ **insertMany avec ordered / unordered**

```
db.Stagiaires.insertMany(
 [
  { prenom: "Ali" },
  { prenom: "Sara" }
 ],
 { ordered: false }
)
```

- insertMany() ajoute plusieurs documents à la collection.
- Par défaut, l'insertion est **ordonnée** (ordered: true) :
  - Si un document provoque une erreur (ex : _id dupliqué), MongoDB **arrête l'insertion** et les documents suivants ne sont pas insérés.
- Avec **ordered**: false :
  - MongoDB continue d'insérer les autres documents même si certains échouent.
  - Utile pour insérer **rapidement plusieurs documents sans bloquer le processus**.

---- Utile pour expliquer le comportement en cas d'erreur

```
// Résultat
// Document 1 : Amina
{
  "_id" : ObjectId("64f1a0011a2b3c4d5e6f7890"), // identifiant généré automatiquement
  "prenom" : "Amina",
  "filiere" : "Dev Digital",
  "niveau" : "1A"
}
// Document 2 : Nidal
{
  "_id" : ObjectId("64f1a0021a2b3c4d5e6f7891"),
```

```
  "prenom" : "Nidal",
  "filiere" : "Cyber Security",
  "niveau" : "2A"
}
// Document 3 : Salim
{
  "_id" : ObjectId("64f1a0031a2b3c4d5e6f7892"),
  "prenom" : "Salim",
  "filiere" : "Full Stack",
  "niveau" : "2A"
}
```

Lecture de documents

1. **findOne** – Retourne le premier document

db.Stagiaires.findOne() *// récupère le premier stagiaire dans la collection*

2. **find** – Retourne tous les documents

db.Stagiaires.find() *// récupère tous les stagiaires*

3. **find avec restriction** – Sélectionner selon des critères

db.Stagiaires.find({"nom": "Alami"}) *// tous les stagiaires dont le nom est "Alami"*

4. **Projection** – Afficher uniquement certains champs

✔ **find avec condition multiple**

```
db.Stagiaires.find({
  filiere: "Dev Digital",
  niveau: "1A"
})
```

db.Stagiaires.find(

```
  {"nom":"Alami"},  // critère de recherche
  {"prenom":1, "option":1, "_id":0} // ne retourne que prénom et option
)
```

✔ **find avec opérateurs de comparaison**

```
db.Stagiaires.find({ age: { $gt: 18 } })
```

Opérateurs essentiels à intégrer :

- $gt (>)
- $gte (≥)
- $lt (<)
- $lte (≤)
- $ne (≠)

✔ **find avec $and (forme explicite)**

```
db.Stagiaires.find({
  $and: [
    { filiere: "Dev Digital" },
    { niveau: "2A" }
  ]
})
```

✔ **find avec $or**

```
db.Stagiaires.find({
  $or: [
    { filiere: "Dev Digital" },
    { filiere: "Cyber Security" }
  ]
})
```

✔ **find avec $in**

```
db.Stagiaires.find({
  niveau: { $in: ["1A", "2A"] }
})
```

✔ **find avec champ absent**

```
db.Stagiaires.find({ telephone: { $exists: false } })
```

PROJECTION – compléments

✔ **Exclure un champ**

```
db.Stagiaires.find({}, { _id: 0 })
```

✔ **Projection combinée (rappel règle)**

```
db.Stagiaires.find({}, { prenom: 1, filiere: 1, _id: 0 })
```

----Nb : **Interdit : mélanger 1 et 0 (sauf _id)**

Fonction distinct

- **distinct** – Liste les valeurs uniques d'un champ

Exemple 1 : options des stagiaires

Supposons que ta collection Stagiaires contient ces documents :

```
{
  "prenom": "Amina",
  "filiere": "Dev Digital",
  "niveau": "1A",
  "option": "Mobile"
}
{
  "prenom": "Nidal",
  "filiere": "Cyber Security",
  "niveau": "2A",
  "option": "Cyber Security"
}
{
  "prenom": "Salim",
  "filiere": "Full Stack",
  "niveau": "2A",
  "option": "FullStack"
}
```

```
{
  "prenom": "Karima",
  "filiere": "Dev Digital",
  "niveau": "1A",
  "option": "Mobile"
}
```

Si on fais :

db.Stagiaires.distinct("option")

MongoDB va parcourir **tous les documents**, prendre la valeur du champ "option" et ne garder **que les valeurs uniques**.

Résultat :

["Mobile", "FullStack", "Cyber Security"]

Même si “Mobile” apparaît deux fois (Amina et Karima), il n’est listé qu’une seule fois.

<u>Exemple 2 : filières</u>

Si tes documents ont un sous-champ "**filiere.intitule**" :

```
{
  "prenom": "Amina",
  "filiere": {"_id": "DD", "intitule": "Dev Digital"},
  "niveau": "1A"
}
{
  "prenom": "Nidal",
  "filiere": {"_id": "CS", "intitule": "Cyber Security"},
  "niveau": "2A"
}
{
  "prenom": "Salim",
  "filiere": {"_id": "FS", "intitule": "Full Stack"},
  "niveau": "2A"
}
```

Et on fais :

db.Stagiaires.distinct("filiere.intitule")

MongoDB regarde **le sous-champ** intitule dans le champ filiere et renvoie uniquement les valeurs uniques :

["Dev Digital", "Cyber Security", "Full Stack"]

Mettre à jour des documents

updateOne

- Met à jour le premier document qui correspond à la condition.
- On peut utiliser $set pour modifier certains champs sans écraser le document entier.

db.Stagiaires.updateOne(

{ "prenom": "Amina" },

{ $**set**: { "niveau": "2A" } }

)

$set permet de **modifier uniquement le champ indiqué** sans toucher aux autres informations du document.

Ici : le champ niveau de la stagiaire Amina est mis à jour pour devenir "2A".

Tous les autres champs (prenom, filiere, age, etc.) restent intacts.

NB : **updateOne** est utile quand on sait qu'il n'y a qu'un seul document à modifier.

✔ **updateOne sans $set (écrasement)**

db.Stagiaires.updateOne(

{ prenom: "Amina" },

{ niveau: "2A" }

)

Ici, MongoDB **remplace entièrement le document** par { niveau: "2A" }.

Tous les autres champs existants (prenom, filiere, age…) **sont supprimés**.

C'est **dangereux** si on voulait juste modifier un champ spécifique.

--- !! **ATTENTION** : **updateOne** SANS set = écrase le document

✔ **Ajouter un champ avec $set**

db.Stagiaires.**updateOne**(

{ prenom: "Amina" },

{ $**set**: { **telephone**: "0600000000" } }

)

$set est un **opérateur MongoDB** qui permet **d'ajouter un nouveau champ** ou **de modifier un champ existant**.

Ici : le champ telephone est ajouté au document Amina avec la valeur "0600000000".

Si le champ existe déjà, sa valeur est remplacée.

✔ **update avec $inc**

db.Stagiaires.**updateOne**(

{ prenom: "Amina" },

{ $**inc**: { age: 1 } }

)

$inc est un **opérateur MongoDB** qui permet **d'incrémenter ou décrémenter une valeur numérique**.

Ici : le champ age du stagiaire Amina est **augmenté de 1**.

Si age n'existe pas, MongoDB le crée et lui attribue la valeur de l'incrément.

✔ **update avec upsert**

db.Stagiaires.**updateOne**(

{ prenom: "Yassine" },

{ $**set**: { filiere: "Dev Digital" } },

{ **upsert**: true }

)

updateOne() : met à jour **le premier document** correspondant au filtre

$set : modifie ou ajoute le champ indiqué

upsert: **true** : si aucun document ne correspond à la condition, MongoDB **crée un nouveau document** avec le contenu du $set et la condition

updateMany

- Met à jour tous les documents qui correspondent à la condition.

db.Stagiaires.updateMany(

{ "filiere.intitule": "Dev Digital" },

{ $set: { "option": "FullStack" } }

)

Cette commande **met à jour tous les documents** dans la collection **Stagiaires** qui correspondent à la condition :

- Condition : "**filiere.intitule**" = "Dev Digital"
- Mise à jour : ajoute ou modifie le champ option avec la valeur "**FullStack**"

NB : **Toujours vérifier avec find() après un updateMany pour éviter des modifications accidentelles.**

Supprimer des documents

deleteOne

- Supprime le premier document correspondant à la condition.

db.Stagiaires.deleteOne(

{ "prenom": "Salim" } *// supprimer le premier stagiaire nommé "Salim"*

)

Cette commande **supprime uniquement le premier document** trouvé dans la collection **Stagiaires** qui correspond à la condition.

- Ici : MongoDB cherche **le premier document où prenom = "Salim"** et le supprime.
- Les autres documents avec le même prénom **restent intacts**.

deleteMany

- Supprime tous les documents correspondant à la condition.

db.Stagiaires.deleteMany(

{ "filiere.intitule": "Cyber Security" } *// supprimer tous les stagiaires de Cyber Security*

)

Cette commande supprime tous les documents où le sous-champ intitule du champ filiere vaut "Cyber Security".

- Ici : MongoDB parcourt tous les documents, vérifie **filiere.intitule**, et supprime ceux qui correspondent.

✔ **delete avec condition multiple**

db.Stagiaires.**deleteMany**({

filiere: "Dev Digital",

niveau: "1A"

})

Cette commande **supprime tous les documents** dans la collection **Stagiaires** qui respectent **toutes les conditions** fournies :

- Ici : tous les stagiaires dont
  - filiere = "Dev Digital" **et**
  - niveau = "1A"

✔ **delete sans condition**

db.Stagiaires.deleteMany({})

----- Supprime tous les documents

Création explicite d'une colllection :

On peut créer la collection **avant d'insérer des documents** avec **createCollection**() :

db.**createCollection**("Stagiaires")

Avantages :

- Permet de **définir des options** comme la taille maximale ou la validation de schéma
- Utile pour **préparer une collection avant de travailler dessus**

Exemple avec options

```
db.createCollection("Stagiaires", {
  capped: true,   // collection à taille fixe
  size: 100000    // taille maximale en octets
})
```

drop

- Supprime la collection entière.

db.Stagiaires.drop() *// supprime complètement la collection Stagiaires*

Tri et filtres

Soit les documents Bson suivants :

```
{
  "_id": ObjectId("1"),
  "nom": "Amina",
  "moy1A": 14.5
},
{
    "_id": ObjectId("2"),
```

```
  "nom": "Salim",
  "moy1A": 12.75
}
```

Tri avec sort

- Permet d'afficher les documents dans un certain ordre.

db.Stagiaires.find().**sort**({ "moy1A": -1 })

Cette commande permet de **trier les documents** de la collection **Stagiaires** selon le champ moy1A :

- -1 → ordre **décroissant** (du plus grand au plus petit)
- 1 → ordre **croissant** (du plus petit au plus grand)

Utile pour classer les stagiaires selon leurs notes, prix, ou toute valeur numérique ou textuelle.

// Affiche tous les stagiaires triés par moyenne décroissante

Limiter le nombre de résultats

- La fonction **limit**(n) permet de limiter les résultats retournés par find.

db.Stagiaires.find().**limit**(2)

// Affiche uniquement les deux premiers documents

✔ **skip (pagination simple)**

db.Stagiaires.**find**().**skip**(2).**limit**(2)

Cette commande permet de :

- **Ignorer** les 2 premiers documents
- Puis **afficher seulement 2 documents** après ceux ignorés

Elle est utilisée pour faire une **pagination simple** (page 1, page 2, etc.).

Combiner tri et filtre

- On peut filtrer et trier ensemble pour une sélection plus précise.

db.Stagiaires.find({ "niveau": "2A" }) *// filtre : niveau 2A*

.sort({ "moy1A": -1 }) *// tri : moyenne décroissante*

.limit(3) *// limite à 3 résultats*

Cette commande permet de :

1. **Sélectionner** uniquement les stagiaires du niveau **2A**
2. **Trier** ces stagiaires selon leur moyenne moy1A du **plus élevé au plus faible**
3. **Limiter** l'affichage aux **3 meilleurs résultats**

Elle est souvent utilisée pour afficher un **classement** ou un **top 3**.

NB : **très utile pour afficher les meilleurs étudiants d'une filière ou pour préparer des rapports.**

Autres méthodes :

✔ **Compter les documents**

db.Stagiaires.**countDocuments**()

Cette commande **compte le nombre total de documents** présents dans la collection **Stagiaires**.

Aucune condition n'est fournie, donc **tous les documents** sont comptés.

✔ **count avec condition**

db.Stagiaires.**countDocuments**({ filiere: "Dev Digital" })

Cette commande **compte le nombre de documents** dans la collection **Stagiaires** qui respectent **la condition donnée**.

Ici, MongoDB compte **combien de stagiaires appartiennent à la filière *Dev Digital***.

OFPPT

Formateur : AARAB HAFID